

RELATÓRIO DE PROJETO

Sistemas Distribuídos

1. Arquitetura do Sistema

O ONE HEALTH é um sistema distribuído de monitorização ambiental em tempo real. A demo inclui vinte sensores IoT simulados distribuídos por várias zonas medem nove variáveis ambientais; as leituras atravessam gateways de fronteira, que as validam, normalizam e analisam, e seguem para um servidor central que as persiste em PostgreSQL e as expõe por uma API REST a uma aplicação web. O sistema organiza-se em cinco camadas:

- Perceção - Sensores IoT simulados (S001–S020) em múltiplos formatos de codificação.
- Edge - Gateways (GW_001 e GW_002) que subscrevem, validam, normalizam e reencaminham.
- Processamento - Serviços gRPC de pré-processamento (C#) e análise ML (Python).
- Persistência e API - Servidor central com PostgreSQL e API REST protegida por JWT.
- Apresentação - Frontend React/TypeScript com dashboards, alarmes e análises.

Stack tecnológica:

Camada	Tecnologia	Justificação
Sensores, Gateways, Servidor, PreProc.	C# / .NET 9	Desempenho, tipagem forte, ecossistema de rede
Análise ML	Python (pandas, numpy, scikit-learn)	Isolation Forest nativo; bom ecossistema de ML
Interface Web	React + TypeScript + Recharts	Componentes reativos, visualização interativa
Mensageria	RabbitMQ (AMQP, topic exchange)	Pub/sub desacoplado com roteamento por zona/tipo
Serviços internos	gRPC (Protocol Buffers)	RPC tipificado e eficiente para processamento em lote
Base de dados	PostgreSQL (TIMESTAMPTZ)	Robustez e indexação para séries temporais
Autenticação	JWT (HMAC-SHA256)	Stateless, sem sessões no servidor

2. Componentes

2.1 Sensor

Classe única instanciada por processo a partir de um diretório de configuração; os sensores partilham o mesmo código. As leituras resultam da sobreposição de um sinal circadiano (24 h), ciclo de ocupação (4 h), microvariação (30 min), derivação Ornstein-Uhlenbeck e ruído gaussiano, sobre um valor base. Eventos ambientais correlacionados (incêndio, tempestade, smog, etc.) afetam múltiplas variáveis em simultâneo. As mensagens podem ser emitidas em cinco formatos: pipe, JSON, XML, QueryString e Hex. Capacidades adicionais incluem streaming de vídeo via UDP e dashboard TUI em tempo real. [--- Ver Anexo A](#)

2.2 Gateway

Node de fronteira (edge). Subscreve o exchange RabbitMQ (por zona geográfica), auto-regista sensores via mensagem HELLO, acumula leituras em blocos por (sensorId, tipo) e envia-os ao PreProcessamento e ao ServicoAnalise via gRPC antes de os reencaminhar ao servidor por TCP persistente orientada a eventos (adormece e acorda conforme há dados para enviar). Avalia alarmes por threshold após normalização, com cooldown de 30 s por sensor por tipo. Mantém uma fila de retry persistente (pendentes.json, máx. 1000 entradas) e um watchdog que marca sensores sem heartbeat como em manutenção, recuperando-os automaticamente quando reestabelece o beat. [--- Ver Anexo B](#)

2.3 PreProcessamento

Serviço gRPC stateless em C#. Deteta automaticamente o formato do payload, converte unidades para a forma canónica (°F/K→°C, mg/m³→µg/m³, fc→lux), valida intervalos e atribui uma pontuação de qualidade entre 0 e 1. Esta avaliação é

afetada pelo número de transformações que alteram os dados (conversões de dados e formato), por anomalias temporais (timestamp errado) e outras condições como dados duplicados (muita pouca variância em dados seguidos).

[--- Ver Anexo C](#)

2.4 ServicoAnalise

Serviço gRPC em Python. Executa quatro operações: ScoreBatch - deteção de anomalias por Isolation Forest com features temporais (sin/cos da hora), retreinado a cada 5 mi, AnalisarZona - estatísticas descritivas por zona; DetectarPadroes - picos, vales e tendência linear; PreviRisco - previsão de curto prazo e índice composto de risco para a saúde. [--- Ver Anexo D](#)

2.5 Servidor Central

Instância única ServerCentral. Recebe blocos dos gateways via TCP e insere-os no PostgreSQL com NpgsqlBatch. Expõe a API REST com JWT. Coordena e apresenta o streaming de vídeo UDP recebido dos sensores. [--- Ver Anexo E](#)

2.6 Frontend

Aplicação React/TypeScript. Dashboard por zona com leituras em tempo real, histórico, anomalias ML, índice de risco, alarmes ativos e gráficos de tendência. Consulta periodicamente a API REST.

3. Comunicação e Protocolos

O sistema usa cinco transportes distintos, cada um adequado ao seu trajeto:

Origem	Destino	Protocolo	Justificação
Sensores	Gateway	RabbitMQ / AMQP	Pub/sub desacoplado; roteamento por zona; durabilidade
Gateway	PreProcessamento	gRPC (50051)	RPC tipificado para normalização em lote
Gateway	ServicoAnalise	gRPC (50052)	Inferência ML estruturada com protobuf
Gateway	Servidor Central	TCP persistente (14000)	Fluxo ordenado e fiável de blocos de dados
Servidor	Gateway	TCP de comando (14001/14002)	Controlo pontual de início/fim de streaming
Sensor	Servidor	UDP (15000, JPEG)	Baixa latência; perda de frames aceitável
Servidor	Frontend	HTTP/REST (8080) + JWT	Integração padrão com clientes web; stateless

4. Fluxo de Dados Ponta a Ponta

O percurso de uma leitura desde o sensor até ao dashboard decorre em sete etapas: **(1)** o sensor publica a leitura no exchange RabbitMQ no formato configurado; **(2)** o gateway recebe e interpreta a mensagem, formatos não-pipe são passados como payload bruto; **(3)** a leitura acumula-se no buffer e é enviada em lote quando o bloco enche ou o timeout expira; **(4)** o lote é enviado ao PreProcessamento via gRPC para deteção de formato, conversão de unidades, validação e pontuação de qualidade; **(5)** os limiares de alarme são verificados centralizadamente sobre os valores normalizados*; **(6)** o lote enriquecido é enviado via TCP ao servidor, que o insere no PostgreSQL; **(7)** o frontend consulta a API REST e o servidor invoca o ServicoAnalise via gRPC para estatísticas, padrões, previsões e anomalias.

*Como o alarme só é verificado depois de o bloco estar completo, este pode ter algum delay. Decidimos fazer este trade-off já que isto não é um sistema de emergências e damos prioridade à verificação de todos os valores incluído os que foram normalizados.

[---Ver Anexo F](#)

5. Objetivos dos Sistemas Distribuídos

Partilha de Recursos

O RabbitMQ é o broker central partilhado por todos os sensores e gateways; o PostgreSQL é a base de dados única acedida pelo servidor central; o ServicoAnalise é um serviço partilhado invocado por ambos os gateways via gRPC. A separação por zonas garante que a partilha não cria contenção entre regiões geográficas.

Transparência

A transparência de acesso está bem implementada, o frontend desconhece a topologia interna e os gateways invocam os serviços gRPC sem conhecer os seus detalhes de implementação. A transparência de localização é parcial: os IPs são configuráveis via variáveis de ambiente, mas não existe descoberta de serviços dinâmica. A transparência de falha cobre apenas falhas transitórias; falhas permanentes do servidor central ou do RabbitMQ são visíveis pelo utilizador.

Abertura

O sistema usa protocolos standard (AMQP, gRPC/protobuf, HTTP/REST, TCP, UDP) e formatos interoperáveis (JSON, XML), permitindo integrar novos componentes sem dependências proprietárias. Adicionar um sensor ou gateway é uma operação de configuração sem alteração de código. A API REST com JWT é um ponto de extensão natural para novos clientes.

Escalabilidade

A escalabilidade de configuração é o ponto forte: adicionar sensores ou gateways requer apenas um diretório de configuração, foi validado com sucesso com 5 gateways e 200 sensores. A escalabilidade de carga é limitada: o servidor central, o PostgreSQL e o ServicoAnalise são instâncias únicas que se tornam bottlenecks com volume crescente. O chunking nos gateways atenua a pressão, mas não elimina o gargalo central.

O instaciamento de ServicoAnalise com um balance loader e o particionamento da base de dados pode atenuar um bocado os bottlenecks.

Tolerância a Falhas

O sistema trata bem falhas transitórias: reconexão automática ao RabbitMQ e ao servidor TCP, fila de retry persistente para lotes não entregues, e watchdog de sensores com recuperação automática por heartbeat. A principal limitação é a ausência de redundância nos componentes centrais, o RabbitMQ, o servidor e o PostgreSQL são pontos únicos de falha cujas quedas paralisam o sistema.

6. Trabalho Futuro e Melhorias

- Base de dados – plugin do Postgres, TimescaleDB com particionamento temporal, hypertables e políticas de retenção.
- ServicoAnalise - persistência dos modelos ML em disco, múltiplas instâncias com load balancer e circuit breaker.
- API REST - migração de HttpListener para Kestrel/ASP.NET Core com caching Redis e suporte a WebSockets.
- Frontend - substituição de polling HTTP por WebSockets para atualizações push genuínas.
- Protocolos IoT - migração do transporte sensor-gateway de AMQP para MQTT, mais adequado a dispositivos com recursos limitados.

Turma PL5

Lucas Silva- AL81362

Rafael Carvalho- al81971

Rafael Ferreira- AL81814

<https://github.com/rafaelf2014/Sistemas-Distribuidos.git>

Anexos

Anexo A - Sensor_DataGen.cs, método GerarBaseline

Geração de leituras por sobreposição de sinais (circadiano, ocupação, microvariação, Ornstein-Uhlenbeck, noise gaussiano) com perfil base por tipo de zona.

```
57 // Gera o valor base de uma variavel combinando varias camadas: ciclo de 24h,
58 // ciclo de 4h, microvariacao de meia hora, deriva e noise gaussiano.
59 // O dia simulado dura 7200 segundos reais (2 horas).
60 private double GerarBaseline(string tipo)
61 {
62     var p = _profiles[ObterZonaTipo()];
63     double h = (DateTime.Now.TimeOfDay.TotalSeconds % 7200.0) / 7200.0 * 24.0;
64
65     double sin24 = Math.Sin(2 * Math.PI / 24.0 * (h - 6));
66     double sin4 = Math.Sin(2 * Math.PI / 4.0 * h);
67     double sin05 = Math.Sin(2 * Math.PI / 0.5 * h);
68
69 // Atualiza a deriva: media movel suave mais um passo aleatorio.
70 double prev = _drift.GetOrAdd(tipo, 0.0);
71 _drift[tipo] = prev * 0.92 + (Random.Shared.NextDouble() * 2 - 1);
72
73 double d = _drift[tipo];
74 double g = Gauss();
75
76 double v = tipo switch
77 {
78     "TEMP" => p.BaseTEMP + 7.0 * sin24 + 1.8 * sin4 + 0.6 * sin05 + g * 0.4 + d * 0.9,
79     "HUM" => p.BaseHUM - 14.0 * sin24 + 2.5 * sin4 + 0.8 * sin05 + g * 0.7 + d * 1.2,
80     "CO2" => p.BaseCO2 + 180.0 * sin24 + 65.0 * sin4 + 22.0 * sin05 + g * 9 + d * 18,
81     "RUIDO" => p.BaseRUIDO + 10.0 * Math.Abs(sin24) + 7.0 * Math.Abs(sin4) + 2.5 * sin05 + g * 1.5 + d * 2.5,
82     "LUMIN" => Math.Max(0, p.BaseLUMIN * Math.Max(0, sin24) + g * 250 + d * 350),
83     "PART" => Math.Max(0, p.BasePART + 7.0 * sin24 + 2.5 * sin4 + g * 1.2 + d * 2),
84     "NO2" => Math.Max(0, p.BaseNO2 + 22.0 * Math.Abs(sin24) + 9.0 * Math.Abs(sin4) + g * 3 + d * 4.5),
85     "O3" => Math.Max(0, p.BaseO3 + 14.0 * Math.Max(0, sin24) + 4.0 * sin4 + g * 2 + d * 3),
86     "WIND" => Math.Max(0, p.BaseWIND + 4.5 * sin24 + 2.0 * (Random.Shared.NextDouble() * 2 - 1) + Math.Abs(g) * 1.5),
87     _ => 50 + g
88 };
89
90 return v;
91 }
```

Anexo B - Gateway.cs, método ScoreEEnviar

Fluxo combinado de normalização (gRPC PreProcessamento), pontuação de anomalia (gRPC ServicoAnalise) e verificação de alarmes, antes do envio TCP ao servidor.

```
623 // Ponto unico por onde passam todos os caminhos de ingestao (pipe e formatos brutos).
624 // Avalia alarmes sobre valores ja normalizados, pede o scoring de anomalias ao
625 // ServicoAnalise e envia o lote ao servidor.
626 async Task ScoreEEnviar(List<LeituraFinal> finais)
627 {
628     for (int i = 0; i < finais.Count; i++)
629     {
630         if (VerificarAlarme(finais[i]))
631             finais[i] = finais[i] with { IsAlarm = true };
632
633         if (_analiseClient != null)
634         {
635             try
636             {
637                 var pedido = new PedidoScoreBatch { GatewayId = _gatewayId };
638                 foreach (var f in finais)
639                     pedido.Leituras.Add(new LeituraScore
640                     {
641                         SensorId = f.SensorId,
642                         Zona = f.Zona,
643                         Tipo = f.TipoDado,
644                         Valor = f.Valor,
645                         Timestamp = f.Timestamp.ToString("yyyy-MM-ddTHH:mm:ssZ")
646                     });
647
648                 var resultado = await _analiseClient.ScoreBatchAsync(pedido);
649
650                 if (resultado.ModeloAquecido)
651                 {
652                     for (int i = 0; i < Math.Min(resultado.Anomalias.Count, finais.Count); i++)
653                     {
654                         var a = resultado.Anomalias[i];
655                         finais[i] = finais[i] with { AnomalyScore = a.Score };
656                         if (a.IsAnomalia)
657                             RegistrarLogEsquerda(
658                                 $"[ML] Anomalia: {finais[i].SensorId} {finais[i].TipoDado} score={a.Score:F2} - {a.Motivo}", true);
659                     }
660                 }
661                 catch (Exception ex) { RegistrarLogEsquerda($"[ML] ScoreBatch falhou: {ex.Message}"); }
662             }
663
664             await EnviarBatchParaServidor(finais);
665         }
666     }
667 }
```

Anexo C - PreProcessamentoService.cs, método Processar

Pipeline de quatro etapas: deteção de formato, conversão de unidades, validação de intervalo e pontuação de qualidade de cada leitura.

```

306 // Valida o intervalo e calcula a pontuacao de qualidade (0 a 1), penalizando
307 // conversoes, valores nos extremos, sensor bloqueado e timestamps implausiveis.
308 private static LeituraProcessada Processar(
309     LeituraBruta leitura, double valor, bool convertido,
310     BatchFlags bf, TsFlags tf)
311 {
312     string tipo = leitura.Tipo.ToUpper();
313     bool valido = true;
314     float qualidade = 1.0f;
315     var obs = new System.Text.StringBuilder();
316
317     void Anotar(string msg)
318     {
319         if (obs.Length > 0) obs.Append("; ");
320         obs.Append(msg);
321     }
322
323     if (convertido) qualidade *= 0.98f;
324
325     if (_intervalos.TryGetValue(tipo, out var intervalo))
326     {
327         if (valor < intervalo.Min || valor > intervalo.Max)
328         {
329             valido = false;
330             qualidade = 0.0f;
331             Anotar($"Valor {Math.Round(valor, 2)} fora de [{intervalo.Min},{intervalo.Max}]");
332         }
333         else
334         {
335             double pos = (valor - intervalo.Min) / (intervalo.Max - intervalo.Min);
336             if (Math.Abs(pos - 0.5) > 0.425) qualidade -= 0.10f;
337         }
338     }
339
340     if (valido)
341     {
342         if (bf.IsBloqueado)
343         {
344             qualidade *= 0.40f;
345             Anotar("Sensor bloqueado - variância zero");
346         }
347         else if (bf.IsSuspeito)
348         {
349             qualidade *= 0.75f;
350             Anotar("Variância suspeita no batch");
351         }
352
353         if (tf.Futuro) { qualidade *= 0.80f; Anotar("Timestamp futuro"); }
354         if (tf.Antigo) { qualidade *= 0.85f; Anotar("Leitura atrasada (>30min)"); }
355         if (tf.Desordem) { qualidade *= 0.90f; Anotar("Desordem temporal"); }
356     }
357
358     qualidade = Math.Max(0.0f, Math.Min(1.0f, qualidade));
359
360     return new LeituraProcessada
361     {
362         GatewayId = leitura.GatewayId,
363         SensorId = leitura.SensorId,
364         Zona = leitura.Zona,
365         Tipo = tipo,
366         ValorNormalizado = Math.Round(valor, 2),
367         UnidadePadrao = _unidadesPadrao.GetValueOrDefault(tipo, leitura.Unidade),
368         Timestamp = leitura.Timestamp,
369         Valido = valido,
370         Observacao = obs.ToString(),
371         Qualidade = qualidade
372     };
373 }
374
    
```

Anexo D - server.py, método ScoreBatch

Deteção de anomalias com Isolation Forest por tipo de dado, usando features [valor, sin(hora), cos(hora)]; retreino automático em background a cada 5 minutos.

```

132 # Pontua um lote de leituras com o modelo de cada tipo. Devolve, por leitura, um score
133 # de anomalia entre 0 e 1 e se e ou nao anomalia. modelo_aquecido indica se ja havia
134 # pelo menos um modelo treinado.
135 def ScoreBatch(self, request, context):
136     por_tipo = defaultdict(list) # tipo -> [(indice, valor, timestamp)]
137     for i, l in enumerate(request.leituras):
138         por_tipo[l.tipo.upper()].append((i, l.valor, l.timestamp))
139
140     resultados = [None] * len(request.leituras)
141     algum_modelo = False
142
143     with _model_lock:
144         snapshot = dict(_models)
145
146     for tipo, items in por_tipo.items():
147         modelo = snapshot.get(tipo)
148         if modelo is None:
149             continue
150         algum_modelo = True
151
152         vals, sin_h, cos_h = [], [], []
153         for _, v, ts in items:
154             try:
155                 dt = datetime.fromisoformat(ts.replace("Z", "+00:00")) if ts else datetime.now(timezone.utc)
156             except Exception:
157                 dt = datetime.now(timezone.utc)
158             if dt.tzinfo is None:
159                 dt = dt.replace(tzinfo=timezone.utc)
160             h = _hora_simulada_de_dt(dt)
161             vals.append(v)
162             sin_h.append(np.sin(2 * np.pi * h / 24.0))
163             cos_h.append(np.cos(2 * np.pi * h / 24.0))
164
165         X = np.column_stack([vals, sin_h, cos_h])
166         try:
167             decisions = modelo.decision_function(X)
168         except Exception:
169             continue
170
171         print(f"[ML] {tipo}: decisions={([round(float(d),3) for d in decisions])}", flush=True)
172         for j in range(len(items)):
173             idx = items[j][0]
174             decision = decisions[j]
175             # decision fica perto de [-0.15, +0.15]: 0.0 e a fronteira.
176             # Mapeia para score: 0.0 -> 0.5, -0.15 -> 0.95, +0.15 -> 0.05.
177             score = float(np.clip(0.5 - (decision * 3.0), 0.0, 1.0))
178             is_anomalia = bool(decision < 0)
179             leitura = request.leituras[idx]
180             resultados[idx] = analysis_pb2.AnomaliaInfo(
181                 sensor_id = leitura.sensor_id,
182                 tipo = leitura.tipo,
183                 score = score,
184                 is_anomalia = is_anomalia,
185                 motivo = f"IF score={score:.2f}" if is_anomalia else ""
186             )
187
188     anomalias = []
189     for i, leitura in enumerate(request.leituras):
190         anomalias.append(resultados[i] if resultados[i] is not None else
191             analysis_pb2.AnomaliaInfo(
192                 sensor_id=leitura.sensor_id, tipo=leitura.tipo,
193                 score=0.0, is_anomalia=False, motivo=""
194             ))
195
196     return analysis_pb2.ResultadoScoreBatch(
197         anomalias=anomalias,
198         modelo_aquecido=algum_modelo
199     )

```

Anexo E - Server.cs, método HandleDataBatch

Receção e deserialização dos blocos TCP dos gateways e inserção em massa no PostgreSQL via NpgsqlBatch.

```

115 // Insere um lote de leituras na base de dados (uma so operacao em batch) e atualiza
116 // o ultima_sync de cada sensor envolvido.
117 async Task<string> HandleDataBatch(JsonElement root, string gatewayId)
118 {
119     if (!root.TryGetProperty("leituras", out var leituras) || leituras.ValueKind != JsonValueKind.Array)
120         return "{\"tipo\":\"ACK_BATCH\",\"status\":\"ERRO\",\"erro\":\"Campo leituras ausente\"}";
121
122     int count = 0;
123     var sensorIdsNoBatch = new System.Collections.Generic.HashSet<string>();
124     try
125     {
126         await using var conn = new NpgsqlConnection(connectionString);
127         await conn.OpenAsync();
128         await using var batch = new NpgsqlBatch(conn);
129
130         foreach (var l in leituras.EnumerateArray())
131         {
132             string sensorId = l.TryGetProperty("sensorId", out var s) ? s.GetString() ?? "" : "";
133             string zona = l.TryGetProperty("zona", out var z) ? z.GetString() ?? "" : "";
134             string tipoDado = l.TryGetProperty("tipoDado", out var td) ? td.GetString() ?? "" : "";
135             double valor = l.TryGetProperty("valor", out var v) ? v.GetDouble() : 0;
136             string tsStr = l.TryGetProperty("timestamp", out var ts) ? ts.GetString() ?? "" : "";
137             double qualidade = l.TryGetProperty("qualidade", out var q) ? q.GetDouble() : 1.0;
138             bool isAlarm = l.TryGetProperty("isAlarm", out var ia) && ia.GetBoolean();
139             double? anomalyScore = l.TryGetProperty("anomalyScore", out var asc) && asc.ValueKind != JsonValueKind.Null
140                 ? asc.GetDouble() : (double?)null;
141
142             DateTime timestamp = DateTime.TryParse(tsStr, null,
143                 DateTimeStyles.AssumeUniversal | DateTimeStyles.AdjustToUniversal,
144                 out DateTime dt) ? dt : DateTime.UtcNow;
145
146             if (isAlarm) RegistrarLog($"[{zona}] ANOMALIA! Sensor: {sensorId} | {tipoDado} = {valor}", true);
147
148             var cmd = new NpgsqlBatchCommand(
149                 "INSERT INTO leituras (sensor_id, gateway_id, zona, tipo_dado, valor, timestamp, is_alarm, qualidade, anomaly_score) " +
150                 "VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)");
151             cmd.Parameters.Add(new NpgsqlParameter { Value = sensorId });
152             cmd.Parameters.Add(new NpgsqlParameter { Value = gatewayId });
153             cmd.Parameters.Add(new NpgsqlParameter { Value = zona });
154             cmd.Parameters.Add(new NpgsqlParameter { Value = tipoDado });
155             cmd.Parameters.Add(new NpgsqlParameter { Value = valor });
156             cmd.Parameters.Add(new NpgsqlParameter { Value = timestamp });
157             cmd.Parameters.Add(new NpgsqlParameter { Value = isAlarm });
158             cmd.Parameters.Add(new NpgsqlParameter { Value = (float)qualidade });
159             cmd.Parameters.Add(new NpgsqlParameter { Value = anomalyScore.HasValue ? (object)(float)anomalyScore.Value : DBNull.Value });
160             batch.BatchCommands.Add(cmd);
161             if (!string.IsNullOrEmpty(sensorId)) sensorIdsNoBatch.Add(sensorId);
162             count++;
163
164             RegistrarLog($"[{zona}] {sensorId} + {tipoDado} = {valor}");
165         }
166
167         if (count > 0)
168         {
169             await batch.ExecuteNonQueryAsync();
170             // Atualiza ultima_sync para o verificador de heartbeat saber que o sensor esta vivo.
171             await using var syncCmd = conn.CreateCommand();
172             syncCmd.CommandText = "UPDATE sensores SET ultima_sync = NOW() WHERE sensor_id = ANY($1)";
173             syncCmd.Parameters.Add(new NpgsqlParameter { Value = sensorIdsNoBatch.ToArray() });
174             await syncCmd.ExecuteNonQueryAsync();
175         }
176     }
177     catch (Exception ex)
178     {
179         RegistrarLog($"Erro batch INSERT: {ex.Message}");
180         return JsonSerializer.Serialize(new { tipo = "ACK_BATCH", status = "ERRO", erro = ex.Message });
181     }
182
183     return $"{{\"tipo\":\"ACK_BATCH\",\"status\":\"OK\",\"count\":{\"count\"}}}\";
184 }

```

Anexo F - Diagrama da Topologia do Sistema

Representação visual da arquitetura completa: sensores, gateways, serviços gRPC, servidor central, base de dados e frontend, com os cinco protocolos de comunicação assinalados (RabbitMQ/AMQP, gRPC, TCP, UDP, HTTP/REST).

